```
!pip install tensorflow==2.10.0
!pip install keras==2.10.0
!pip install h5py==3.7.0
!pip install efficientnet
```

```
Requirement already satisfied: tensorflow==2.10.0 in /usr/local/lib/python3.10/dist-packages (2.10.0)
Requirement already satisfied: absl-py>=1.0.0 in /usr/local/lib/python3.10/dist-packages (from tensorflow==2.10.0) (1.4.0)
Requirement already satisfied: astunparse>=1.6.0 in /usr/local/lib/python3.10/dist-packages (from tensorflow==2.10.0) (1.6.3)
Requirement already satisfied: flatbuffers>=2.0 in /usr/local/lib/python3.10/dist-packages (from tensorflow==2.10.0) (23.5.26)
Requirement already satisfied: gast<=0.4.0,>=0.2.1 in /usr/local/lib/python3.10/dist-packages (from tensorflow==2.10.0) (0.4.0)
Requirement already satisfied: google-pasta>=0.1.1 in /usr/local/lib/python3.10/dist-packages (from tensorflow==2.10.0) (0.2.0)
Requirement already satisfied: grpcio<2.0,>=1.24.3 in /usr/local/lib/python3.10/dist-packages (from tensorflow==2.10.0) (1.62.0)
Requirement already satisfied: h5py>=2.9.0 in /usr/local/lib/python3.10/dist-packages (from tensorflow==2.10.0) (3.7.0)
Requirement already satisfied: keras<2.11,>=2.10.0 in /usr/local/lib/python3.10/dist-packages (from tensorflow==2.10.0) (2.10.0)
Requirement already satisfied: keras-preprocessing>=1.1.1 in /usr/local/lib/python3.10/dist-packages (from tensorflow==2.10.0) (
Requirement already satisfied: libclang>=13.0.0 in /usr/local/lib/python3.10/dist-packages (from tensorflow==2.10.0) (16.0.6)
Requirement already satisfied: numpy>=1.20 in /usr/local/lib/python3.10/dist-packages (from tensorflow==2.10.0) (1.25.2)
Requirement already satisfied: opt-einsum>=2.3.2 in /usr/local/lib/python3.10/dist-packages (from tensorflow==2.10.0) (3.3.0)
Requirement already satisfied: packaging in /usr/local/lib/python3.10/dist-packages (from tensorflow==2.10.0) (23.2)
Requirement already satisfied: protobuf<3.20,>=3.9.2 in /usr/local/lib/python3.10/dist-packages (from tensorflow==2.10.0) (3.19.
Requirement already satisfied: setuptools in /usr/local/lib/python3.10/dist-packages (from tensorflow==2.10.0) (67.7.2)
Requirement already satisfied: six>=1.12.0 in /usr/local/lib/python3.10/dist-packages (from tensorflow==2.10.0) (1.16.0)
Requirement already satisfied: tensorboard<2.11,>=2.10 in /usr/local/lib/python3.10/dist-packages (from tensorflow==2.10.0) (2.1
Requirement already satisfied: tensorflow-io-gcs-filesystem>=0.23.1 in /usr/local/lib/python3.10/dist-packages (from tensorflow=
Requirement already satisfied: tensorflow-estimator<2.11,>=2.10.0 in /usr/local/lib/python3.10/dist-packages (from tensorflow==2
Requirement already satisfied: termcolor>=1.1.0 in /usr/local/lib/python3.10/dist-packages (from tensorflow==2.10.0) (2.4.0)
Requirement already satisfied: typing-extensions>=3.6.6 in /usr/local/lib/python3.10/dist-packages (from tensorflow==2.10.0) (4.
Requirement already satisfied: wrapt>=1.11.0 in /usr/local/lib/python3.10/dist-packages (from tensorflow==2.10.0) (1.14.1)
Requirement already satisfied: wheel<1.0,>=0.23.0 in /usr/local/lib/python3.10/dist-packages (from astunparse>=1.6.0->tensorflow
Requirement already satisfied: google-auth<3,>=1.6.3 in /usr/local/lib/python3.10/dist-packages (from tensorboard<2.11,>=2.10->t
Requirement already satisfied: google-auth-oauthlib<0.5,>=0.4.1 in /usr/local/lib/python3.10/dist-packages (from tensorboard<2.1
Requirement already satisfied: markdown>=2.6.8 in /usr/local/lib/python3.10/dist-packages (from tensorboard<2.11,>=2.10->tensorf
Requirement already satisfied: requests<3,>=2.21.0 in /usr/local/lib/python3.10/dist-packages (from tensorboard<2.11,>=2.10->ten
Requirement already satisfied: tensorboard-data-server<0.7.0,>=0.6.0 in /usr/local/lib/python3.10/dist-packages (from tensorboar
Requirement already satisfied: tensorboard-plugin-wit>=1.6.0 in /usr/local/lib/python3.10/dist-packages (from tensorboard<2.11,>
Requirement already satisfied: werkzeug>=1.0.1 in /usr/local/lib/python3.10/dist-packages (from tensorboard<2.11,>=2.10->tensorf
Requirement already satisfied: cachetools<6.0,>=2.0.0 in /usr/local/lib/python3.10/dist-packages (from google-auth<3,>=1.6.3->te
Requirement already satisfied: pyasn1-modules>=0.2.1 in /usr/local/lib/python3.10/dist-packages (from google-auth<3,>=1.6.3->ten
Requirement already satisfied: rsa<5,>=3.1.4 in /usr/local/lib/python3.10/dist-packages (from google-auth<3,>=1.6.3->tensorboard
Requirement already satisfied: requests-oauthlib>=0.7.0 in /usr/local/lib/python3.10/dist-packages (from google-auth-oauthlib<0.
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.10/dist-packages (from requests<3,>=2.21.0->te
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.10/dist-packages (from requests<3,>=2.21.0->tensorboard<2.
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.10/dist-packages (from requests<3,>=2.21.0->tensorbo
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.10/dist-packages (from requests<3,>=2.21.0->tensorbo
Requirement already satisfied: MarkupSafe>=2.1.1 in /usr/local/lib/python3.10/dist-packages (from werkzeug>=1.0.1->tensorboard<2
Requirement already satisfied: pyasn1<0.6.0,>=0.4.6 in /usr/local/lib/python3.10/dist-packages (from pyasn1-modules>=0.2.1->goog
Requirement already satisfied: oauthlib>=3.0.0 in /usr/local/lib/python3.10/dist-packages (from requests-oauthlib>=0.7.0->google
Requirement already satisfied: keras==2.10.0 in /usr/local/lib/python3.10/dist-packages (2.10.0)
Requirement already satisfied: h5py==3.7.0 in /usr/local/lib/python3.10/dist-packages (3.7.0)
Requirement already satisfied: numpy>=1.14.5 in /usr/local/lib/python3.10/dist-packages (from h5py==3.7.0) (1.25.2)
Requirement already satisfied: efficientnet in /usr/local/lib/python3.10/dist-packages (1.1.1)
Requirement already satisfied: keras-applications<=1.0.8,>=1.0.7 in /usr/local/lib/python3.10/dist-packages (from efficientnet)
Requirement already satisfied: scikit-image in /usr/local/lib/python3.10/dist-packages (from efficientnet) (0.19.3)
Requirement already satisfied: numpy>=1.9.1 in /usr/local/lib/python3.10/dist-packages (from keras-applications<=1.0.8,>=1.0.7->
Requirement already satisfied: h5py in /usr/local/lib/python3.10/dist-packages (from keras-applications<=1.0.8,>=1.0.7->efficien
Requirement already satisfied: scipy>=1.4.1 in /usr/local/lib/python3.10/dist-packages (from scikit-image->efficientnet) (1.11.4
Requirement already satisfied: networkx>=2.2 in /usr/local/lib/python3.10/dist-packages (from scikit-image->efficientnet) (3.2.1
Requirement already satisfied: pillow!=7.1.0,!=7.1.1,!=8.3.0,>=6.1.0 in /usr/local/lib/python3.10/dist-packages (from scikit-ima
Requirement already satisfied: imageio>=2.4.1 in /usr/local/lib/python3.10/dist-packages (from scikit-image->efficientnet) (2.31
Requirement already satisfied: tifffile>=2019.7.26 in /usr/local/lib/python3.10/dist-packages (from scikit-image->efficientnet)
Requirement already satisfied: PyWavelets>=1.1.1 in /usr/local/lib/python3.10/dist-packages (from scikit-image->efficientnet) (1
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.10/dist-packages (from scikit-image->efficientnet) (23.
```

```
from google.colab import drive
drive.mount('/content/drive')
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).
```

```
import os
import cv2
import keras
import pickle
import imblearn
import numpy as np
import pandas as pd
import seaborn as sns
from PIL import Image
import seaborn as sns
import tensorflow as tf
from keras import applications
from keras.layers import Dense
from keras import backend as K
import matplotlib.pyplot as plt
from keras.utils import np_utils
from sklearn.manifold import TSNE
from tqdm import tqdm,tqdm_notebook
from prettytable import PrettyTable
from keras.models import Sequential
from keras.constraints import maxnorm
from tensorflow.keras import optimizers
from imblearn.over_sampling import SMOTE
from tensorflow.keras import applications
from keras.models import Model, load_model
from tensorflow.keras.optimizers import Adam
from keras import optimizers,Model,Sequential
from efficientnet.keras import EfficientNetB2
from sklearn.metrics import classification_report
from keras.layers.convolutional import MaxPooling2D
from sklearn.model_selection import train_test_split
from keras.layers import Flatten, BatchNormalization
from tensorflow.keras import regularizers, optimizers
from keras.preprocessing.image import ImageDataGenerator
from keras.layers.convolutional import Conv2D, SeparableConv2D
from keras.layers import Dropout, GaussianNoise, GaussianDropout
from tensorflow.keras.callbacks import EarlyStopping,ReduceLROnPlateau
from sklearn.metrics import confusion_matrix, cohen_kappa_score,accuracy_score
from keras.layers import Input,GlobalAveragePooling2D,Dropout,Dense,Activation
from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau, TensorBoard, ModelCheckpoint
from tensorflow.keras.preprocessing.image import ImageDataGenerator,img_to_array,array_to_img,load_img
```

```
# Basic Libs..
import warnings
warnings.filterwarnings("ignore")
```

```
import tensorflow as tf
physical_devices = tf.config.list_physical_devices('GPU')
```

```
%cp /content/drive/MyDrive/diabetic_retinopathy_image_classification/train_valid_test_df/df_train_train -d /content
%cp /content/drive/MyDrive/diabetic_retinopathy_image_classification/train_valid_test_df/df_train_test -d /content
```

```
file = open('df_train_train', 'rb')
df_train_train = pickle.load(file)
file.close()

file = open('df_train_test', 'rb')
df_train_test = pickle.load(file)
file.close()
```

```
import os
len(os.listdir('/content/drive/MyDrive/diabetic_retinopathy_image_classification/Data/train_images_resized_preprocessed'))
```

```
2929
```

```
df_train_train.head()
```

	id_code	diagnosis	file_path	file_name
1066	4c52922f3bfd	2	/content/drive/MyDrive/Diabetic_Retinopathy_cl...	4c52922f3bfd.png
1690	77a1f1398fdb	0	/content/drive/MyDrive/Diabetic_Retinopathy_cl...	77a1f1398fdb.png
871	3f47f83217b5	2	/content/drive/MyDrive/Diabetic_Retinopathy_cl...	3f47f83217b5.png
3556	f7735b6d47f7	0	/content/drive/MyDrive/Diabetic_Retinopathy_cl...	f7735b6d47f7.png
1946	891329021e12	0	/content/drive/MyDrive/Diabetic_Retinopathy_cl...	891329021e12.png

```
df_train_train['diagnosis'].value_counts()
```

```
0    1440
2     807
1     289
4     237
3     156
Name: diagnosis, dtype: int64
```

```
df_train_train.shape[0]
```

```
2929
```

```
df_train_train['file_name']
```

```
1066    4c52922f3bfd.png
1690    77a1f1398fdb.png
871     3f47f83217b5.png
3556    f7735b6d47f7.png
1946    891329021e12.png
                ...
3437    eed4afc8ec83.png
1152    50ddd7d976df.png
141     0a85a1e8f9e9.png
2542    afc744fad65e.png
3424    ee059945b08a.png
Name: file_name, Length: 2929, dtype: object
```

```
df_train_train = df_train_train.reset_index()
```

```
X_train = []
y_train = []
image_size = 128
for i in range(df_train_train.shape[0]):
  print(df_train_train['file_name'][i])
  image = cv2.imread(os.path.join('/content/drive/MyDrive/diabetic_retinopathy_image_classification/Data/train_images_resized_preproces
  image = cv2.resize(image, (image_size, image_size))
  X_train.append(image)
  y_train.append(df_train_train['diagnosis'][i])
```

```
4c52922f3bfd.png
77a1f1398fdb.png
3f47f83217b5.png
f7735b6d47f7.png
891329021e12.png
55fd453001cc.png
8d3d67661620.png
7b9d519cbd66.png
7d0a871c45db.png
a86128b601a7.png
d66ccb75ada1.png
4e85aa647534.png
6d259b5b4c76.png
135575dc57c9.png
c7b622ec8104.png
b92eacd1392a.png
cd9e2190c73f.png
756b0d6488bb.png
aea59ebec445.png
f762c272c522.png
e4730ddde408.png
a443c4fd489c.png
b7a1bb106051.png
6fb656d506b2.png
709784f7fcc2.png
2b3a4a81d748.png
d866c26d76f0.png
f09fd9433dff.png
02cd34a85b24.png
57f933d3d7c7.png
b4e15102cd7a.png
789434d095d1.png
999115d9386b.png
ea7e21bab610.png
27fca9f12b3c.png
c58e5c0c5b33.png
7bc00e58d419.png
c57c164bca05.png
8d8aca52c07b.png
2c2aa057afc5.png
6e092caa065f.png
33b91def2035.png
7b691d9ced34.png
d844a7252f4e.png
24f3e70f0419.png
```

```
a7b03e58a6e1.png
13d411c85ffd.png
222f3ee3a1e8.png
4e54ccfd49b2.png
9e2a8135f471.png
9122b31414d3.png
7cc4b7aabe04.png
e8ddfc9709ce.png
356304d15a5c.png
5633ced07d8e.png
fda39982a810.png
18621b9ca978.png
```

```
df_train_test = df_train_test.reset_index()
```

```
X_test = []
y_test = []
image_size = 128
for i in range(df_train_test.shape[0]):
  print(df_train_test['file_name'][i])
  image = cv2.imread(os.path.join('/content/drive/MyDrive/diabetic_retinopathy_image_classification/Data/test_images_resized_preprocesse
  image = cv2.resize(image, (image_size, image_size))
  X_test.append(image)
  y_test.append(df_train_test['diagnosis'][i])
X_test = np.array(X_test)
y_test = np.array(y_test)
```

```
803120c5d287.png
1c578b72d7b3.png
0124dffecf29.png
66366a90d1ef.png
a386ec9aabde.png
0924cec998fa.png
0304bedad8fe.png
33ffddea8c6e.png
fc1b1841eadf.png
d6f36ec5564a.png
922586d86cd8.png
ff344e5c9341.png
3e61703b5ab2.png
2cdcc910778d.png
c7c0470bcf87.png
dd3176bacfe2.png
9a78c6a7b1c2.png
49a4765f8822.png
ac17cc18a994.png
cae33655ca00.png
3cdef7c591cc.png
47b756014447.png
5ce5eeaf757a.png
d4583e9525dc.png
c6a8f8f998a2.png
6d3d1fe6c32a.png
29f44aea93a4.png
974c7d7b9c64.png
76516f828d88.png
ff4832d55461.png
1006345f70b7.png
6f4719c6bb4b.png
10de500cf0c5.png
de778495a1cd.png
88e4399d207c.png
33b893e18eb3.png
81704925f759.png
55f7f018c61c.png
b0cc9f8d06e4.png
66bae1ba227f.png
1da4a17c18c9.png
44271f3cb18f.png
45369821d6a3.png
d911dd40c63b.png
28f93cad89c5.png
5a444c32cd9a.png
18d8fdb140b7.png
624fb7317106.png
82f2784ead76.png
80b5697f2a5e.png
8f06ca4642bd.png
c1896142a20a.png
58059e73d2d4.png
0ad7f631dedb.png
a5bb85afc6e8.png
07751b94a88a.png
5bda2ed09e62.png
66b88a4bc474.png
```

```
X_train = np.array(X_train)
y_train = np.array(y_train)
```

```
X_train.shape
```

```
(2929, 128, 128, 3)
```

```
X_train = X_train.reshape(2929, image_size * image_size * 3)
X_train.shape
```

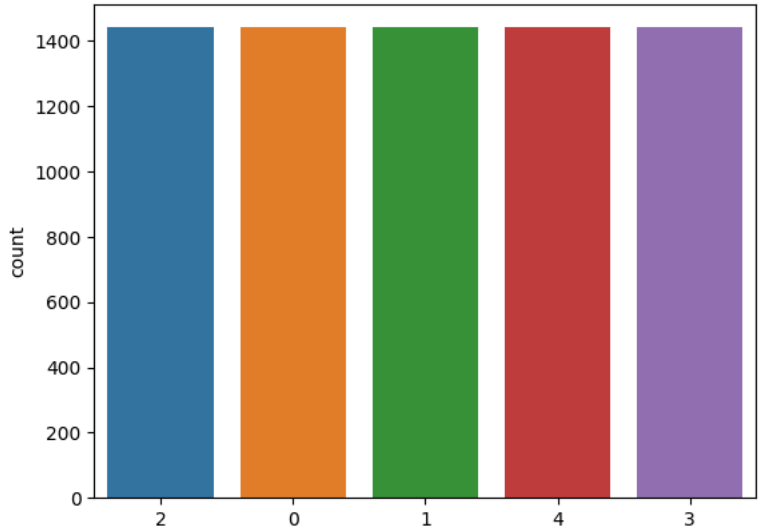
```
(2929, 49152)
```

```
oversample = SMOTE()
X_train, y_train = oversample.fit_resample(X_train, y_train)
```

```
X_train = X_train.reshape(-1,image_size,image_size,3)
```

```
sns.countplot(x=y_train, palette=sns.color_palette())
```

```
<Axes: ylabel='count'>
```




```
X_train, X_val, y_train, y_val = train_test_split(X_train,y_train,test_size=0.1)
```

```
print(X_train.shape)
print(X_val.shape)
print(X_test.shape)
```

```
(6480, 128, 128, 3)
(720, 128, 128, 3)
(733, 128, 128, 3)
```

```
y_train = y_train.astype('int')
y_val = y_val.astype('int')
y_test = y_test.astype('int')
```

```
# Model parameters
BATCH_SIZE = 32
EPOCHS = 10
WARMUP_EPOCHS = 2
LEARNING_RATE = 1e-4
WARMUP_LEARNING_RATE = 1e-3
HEIGHT = 128
WIDTH = 128
CANAL = 3
N_CLASSES = df_train_train['diagnosis'].nunique()
ES_PATIENCE = 5
RLROP_PATIENCE = 3
DECAY_DROP = 0.5
```

CNN Models

[ ] *5 cells hidden*

Resnet50 model

[ ] ↳ *4 cells hidden*

EfficientNetB2

```
base_model = EfficientNetB2(weights='imagenet', include_top=False, input_shape=(image_size, image_size, 3))

model = base_model.output
model = GlobalAveragePooling2D()(model)
model = Dense(5, activation='softmax')(model)
model = Model(inputs = base_model.input, outputs=model)

model.compile(loss='sparse_categorical_crossentropy', optimizer='Adam', metrics=['accuracy'])

model.summary()
```

```
Downloading data from https://github.com/Callidior/keras-applications/releases/download/efficientnet/efficientnet-b2_weights_tf_
31936256/31936256 [==============================] - 0s 0us/step
Model: "model"
__________________________________________________________________________________________________
 Layer (type)                   Output Shape         Param #     Connected to
==================================================================================================
 input_1 (InputLayer)           [(None, 128, 128, 3  0           []
                                )]

 stem_conv (Conv2D)             (None, 64, 64, 32)   864         ['input_1[0][0]']

 stem_bn (BatchNormalization)   (None, 64, 64, 32)   128         ['stem_conv[0][0]']

 stem_activation (Activation)   (None, 64, 64, 32)   0           ['stem_bn[0][0]']

 block1a_dwconv (DepthwiseConv2  (None, 64, 64, 32)  288         ['stem_activation[0][0]']
 D)

 block1a_bn (BatchNormalization  (None, 64, 64, 32)  128         ['block1a_dwconv[0][0]']
 )

 block1a_activation (Activation  (None, 64, 64, 32)  0           ['block1a_bn[0][0]']
 )

 block1a_se_squeeze (GlobalAver  (None, 32)          0           ['block1a_activation[0][0]']
 agePooling2D)

 block1a_se_reshape (Reshape)   (None, 1, 1, 32)     0           ['block1a_se_squeeze[0][0]']

 block1a_se_reduce (Conv2D)     (None, 1, 1, 8)      264         ['block1a_se_reshape[0][0]']

 block1a_se_expand (Conv2D)     (None, 1, 1, 32)     288         ['block1a_se_reduce[0][0]']

 block1a_se_excite (Multiply)   (None, 64, 64, 32)   0           ['block1a_activation[0][0]',
                                                                  'block1a_se_expand[0][0]']

 block1a_project_conv (Conv2D)  (None, 64, 64, 16)   512         ['block1a_se_excite[0][0]']

 block1a_project_bn (BatchNorma  (None, 64, 64, 16)  64          ['block1a_project_conv[0][0]']
 lization)

 block1b_dwconv (DepthwiseConv2  (None, 64, 64, 16)  144         ['block1a_project_bn[0][0]']
 D)

 block1b_bn (BatchNormalization  (None, 64, 64, 16)  64          ['block1b_dwconv[0][0]']
 )

 block1b_activation (Activation  (None, 64, 64, 16)  0           ['block1b_bn[0][0]']
 )

 block1b_se_squeeze (GlobalAver  (None, 16)          0           ['block1b_activation[0][0]']
 agePooling2D)

 block1b_se_reshape (Reshape)   (None, 1, 1, 16)     0           ['block1b_se_squeeze[0][0]']

 block1b_se_reduce (Conv2D)     (None, 1, 1, 4)      68          ['block1b_se_reshape[0][0]']
```

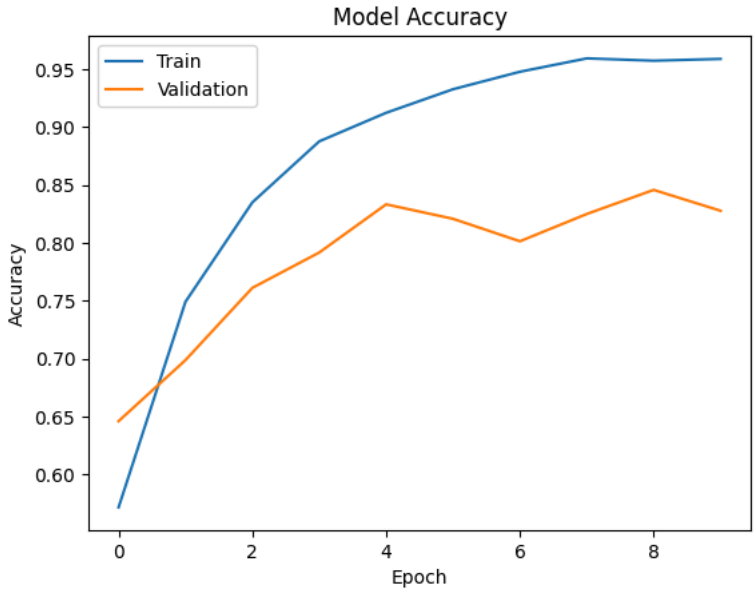
```
history = model.fit(X_train, y_train, epochs=EPOCHS, validation_data=(X_val,y_val), batch_size=BATCH_SIZE, shuffle=True, verbose=1)
```

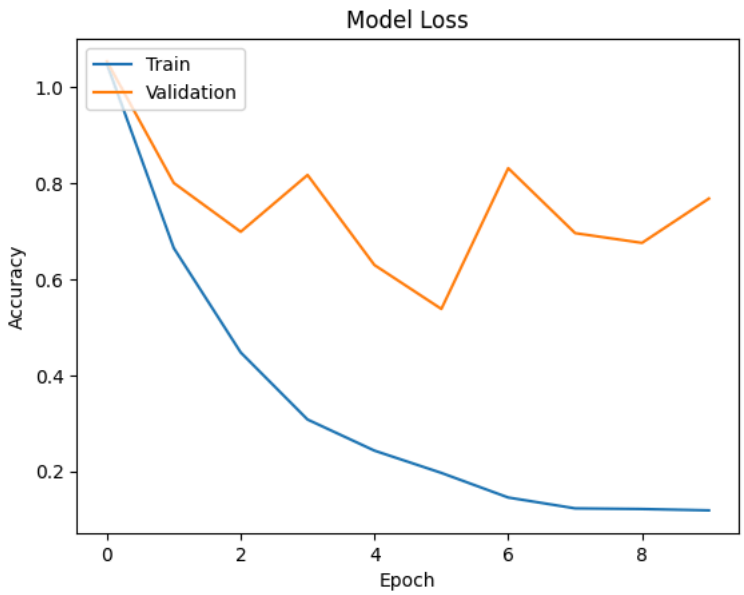
```
Epoch 1/10
203/203 [==============================] - 784s 4s/step - loss: 1.0489 - accuracy: 0.5713 - val_loss: 1.0541 - val_accuracy: 0.6458
Epoch 2/10
203/203 [==============================] - 761s 4s/step - loss: 0.6652 - accuracy: 0.7491 - val_loss: 0.8008 - val_accuracy: 0.6986
```

```
Epoch 3/10
203/203 [==============================] - 751s 4s/step - loss: 0.4479 - accuracy: 0.8350 - val_loss: 0.6992 - val_accuracy: 0.7611
Epoch 4/10
203/203 [==============================] - 767s 4s/step - loss: 0.3080 - accuracy: 0.8878 - val_loss: 0.8175 - val_accuracy: 0.7917
Epoch 5/10
203/203 [==============================] - 769s 4s/step - loss: 0.2432 - accuracy: 0.9125 - val_loss: 0.6299 - val_accuracy: 0.8333
Epoch 6/10
203/203 [==============================] - 772s 4s/step - loss: 0.1969 - accuracy: 0.9329 - val_loss: 0.5384 - val_accuracy: 0.8208
Epoch 7/10
203/203 [==============================] - 758s 4s/step - loss: 0.1456 - accuracy: 0.9480 - val_loss: 0.8315 - val_accuracy: 0.8014
Epoch 8/10
203/203 [==============================] - 782s 4s/step - loss: 0.1230 - accuracy: 0.9596 - val_loss: 0.6963 - val_accuracy: 0.8250
Epoch 9/10
203/203 [==============================] - 761s 4s/step - loss: 0.1216 - accuracy: 0.9576 - val_loss: 0.6760 - val_accuracy: 0.8458
Epoch 10/10
203/203 [==============================] - 772s 4s/step - loss: 0.1188 - accuracy: 0.9591 - val_loss: 0.7683 - val_accuracy: 0.8278
```

```
model.save('/content/drive/MyDrive/diabetic_retinopathy_image_classification/Models/EfficientNetB2.h5')
```

```
plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])
plt.title('Model Accuracy')
plt.ylabel('Accuracy')
plt.xlabel('Epoch')
plt.legend(['Train', 'Validation'], loc='upper left')
plt.gca().ticklabel_format(axis='both', style='plain', useOffset=False)
plt.show()

plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])
plt.title('Model Loss')
plt.ylabel('Accuracy')
plt.xlabel('Epoch')
plt.legend(['Train', 'Validation'], loc='upper left')
plt.gca().ticklabel_format(axis='both', style='plain', useOffset=False)
plt.show()
```

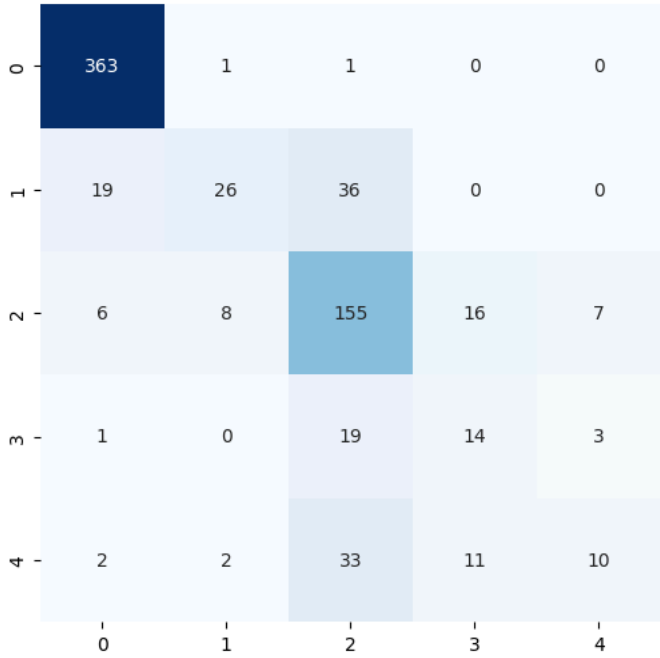





```
pred = model.predict(X_test)
pred = np.argmax(pred,axis=1)
print(accuracy_score(y_test,pred))

cmat = confusion_matrix(y_test,pred)
plt.figure(figsize=(6,6))
sns.heatmap(cmat, annot = True, cbar = False, cmap='Blues', fmt="d");

print(classification_report(y_test,pred))

from sklearn.metrics import precision_recall_fscore_support
res = []
for l in range(N_CLASSES):
     prec,recall,_,_ = precision_recall_fscore_support(y_test==l,
                                                       pred==l,
                                                       pos_label=True,average=None)
     res.append([l,recall[0],recall[1]])

pd.DataFrame(res,columns = ['class','sensitivity','specificity'])
```

```
23/23 [==============================] - 21s 851ms/step
0.7748976807639836
              precision    recall  f1-score   support

           0       0.93      0.99      0.96       365
           1       0.70      0.32      0.44        81
           2       0.64      0.81      0.71       192
           3       0.34      0.38      0.36        37
           4       0.50      0.17      0.26        58

    accuracy                           0.77       733
   macro avg       0.62      0.53      0.55       733
weighted avg       0.76      0.77      0.75       733
```

	class	sensitivity	specificity
0	0	0.923913	0.994521
1	1	0.983129	0.320988
2	2	0.835490	0.807292
3	3	0.961207	0.378378
4	4	0.985185	0.172414



Efficientnet is the best model

Testing

```
IMG_SIZE  = 512
image_size = 128
def crop_image_from_gray(img,tol=7):
    if img.ndim ==2:
        mask = img>tol
        return img[np.ix_(mask.any(1),mask.any(0))]
    elif img.ndim==3:
        gray_img = cv2.cvtColor(img, cv2.COLOR_RGB2GRAY)
        mask = gray_img>tol

        check_shape = img[:,:,0][np.ix_(mask.any(1),mask.any(0))].shape[0]
        if (check_shape == 0): # image is too dark so that we crop out everything,
            return img # return original image
        else:
            img1=img[:,:,0][np.ix_(mask.any(1),mask.any(0))]
            img2=img[:,:,1][np.ix_(mask.any(1),mask.any(0))]
            img3=img[:,:,2][np.ix_(mask.any(1),mask.any(0))]
    #         print(img1.shape,img2.shape,img3.shape)
            img = np.stack([img1,img2,img3],axis=-1)
    #         print(img.shape)
        return img


def circle_crop(img, sigmaX = 30):
    """
    Create circular crop around image centre
    """
    img = crop_image_from_gray(img)
    img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)


filename = '/content/drive/MyDrive/diabetic_retinopathy_image_classification/Data/test_images/006efc72b638.png'
preprocess_image(filename)


    r = np.amin((x,y))
filename = 'inputimage.png'
image = cv2.imread(filename)
image = img_to_array(image)
image = cv2.resize(image, (image_size, image_size))
image = np.array([image])
    return img
```